

Homework 3

Student Name: Lyon Kee

1. Word Embedding (50pts)

- 1.1. Paste your implementation of document representation and cosine similarity calculation. Report the training time of the word embedding model.

Training time: 28.406258821487427s

```
import time
from gensim.models import Word2Vec

start = time.time()
model = Word2Vec(sentences=dataset, min_count=1, workers=4, epochs=10)
end = time.time()
print(f"Word embedding model training time: {end - start}")
model.save("word2vec.model")

import numpy as np

def cosine_similarity(V_q,V_d):
    return V_q @ (V_d/ np.linalg.norm(V_d))

queries = [
    "general chicken",
    "fried chicken",
    "BBQ sandwiches",
    "mashed potatoes",
    "Grilled Shrimp Salad",
    "lamb Shank",
    "Pepperoni pizza",
    "brussel sprout salad",
    "FRIENDLY STAFF",
    "Grilled Cheese"
]
processed_queries = []
for query in queries:
    # Tokenize by word
    tokens = nltk.word_tokenize(query)
    # # Stopword removal
    tokens = [word for word in tokens if not word in stop_words]
    # Token stemming
```

```

        tokens = [nltk.stem.PorterStemmer().stem(token) for token in tokens]
        processed_queries.append(tokens)
    print(processed_queries)

# Compute all doc V_d
print("computing all wv for all docs")
for restaurant in yelp_restaurants:
    for review in restaurant['Reviews']:
        if len(review['NormalizedContent']) != 0:
            review['normalized_wv'] = np.mean([model.wv[word] for word in review['NormalizedContent']])

print("Top 3 document for each query:")
for query in processed_queries:
    doc_similarity = []
    V_q = np.mean([model.wv[word] for word in query], axis=0)
    for restaurant in yelp_restaurants:
        for review in restaurant['Reviews']:
            if len(review.get('normalized_wv', [])) != 0:
                # if review.get('normalized_wv', None) != None:
                    doc_similarity.append((np.absolute(cosine_similarity(V_q, review['normalized_wv']))))
    print(f'query: {query}')
    for similarity, doc in sorted(doc_similarity, reverse=True)[:3]:
        print(f'Similarity: {similarity}')
        print(doc)

```

- 1.2. For the top 3 documents of each query, print the document and its cosine similarity score.

```

query: ['gener', 'chicken']
Similarity: 10.776718139648438
Chicken and jambalya so good.
Similarity: 10.280783653259277
I was a waffle & chicken virgin & I am now addicted. The food is amazing!!! The
Similarity: 10.183368682861328
The chicken here is awesome.
query: ['fri', 'chicken']
Similarity: 14.418567657470703
Fried Chicken. Period.
Similarity: 13.909762382507324
The Fried Chicken is amazing!!
Similarity: 13.10723876953125
Love the fried chicken!! The waffles is just ok but I had to have more chicken
query: ['bbq', 'sandwich']
Similarity: 9.1178617477417
Good pastrami sandwiches! =)

```

Similarity: 9.105042457580566
 BEST PASTRAMI SANDWICH EVER!!! I'm not a big pastrami person...but this sandw
 Similarity: 9.022531509399414
 Their most popular sandwich (pastrami on rye with swiss, cole slaw, and Russian
 query: ['mash', 'potato']
 Similarity: 12.951154708862305
 Mashed potatoes on pizza? Sign me up!
 Similarity: 12.610912322998047
 Potato pizza...mmmmmmmmmmmmmmmmmmmm
 Similarity: 12.58354663848877
 I love that they put the giant sprig of rosemary on the mashed potatoes
 query: ['grill', 'shrimp', 'salad']
 Similarity: 11.27469253540039
 Squid salad and seabass on papaya salad. YUM!
 Similarity: 9.896100997924805
 Loved Guiliana's salad, the spaghetti, and polenta!
 Similarity: 9.614485740661621
 The roasted beet salad is superb; the crab and corn bisque was great. Everyone
 query: ['lamb', 'shank']
 Similarity: 9.995795249938965
 The lamb I order was smelly!!
 Similarity: 9.632453918457031
 3* Buffalo Frog's Legs3* Meatballs3* Seared Maine Scallops4* Black Truffle Gnoc
 Similarity: 9.01131820678711
 The pulled pork tacos are DIVINE!!!!!!
 query: ['pepperoni', 'pizza']
 Similarity: 11.835494041442871
 Posh pizza
 Similarity: 11.644047737121582
 Best deep dish pizza in my book!! We just had a regular pepperoni deep dish piz
 Similarity: 11.347063064575195
 O.M.G.Get the pan pizza.
 query: ['brussel', 'sprout', 'salad']
 Similarity: 13.189217567443848
 Yummy....get the brussel sprouts with brie and the golden beet salad with goat
 Similarity: 11.605668067932129
 Squid salad and seabass on papaya salad. YUM!
 Similarity: 10.827167510986328
 Loved Guiliana's salad, the spaghetti, and polenta!
 query: ['friendly', 'staff']
 Similarity: 14.928730964660645
 love it! staff is so friendly and absolutely no hipster attitude. refreshing!
 Similarity: 14.87571907043457
 Good food, friendly staff, relaxing atmosphere.
 Similarity: 14.738070487976074

```
Great food & service. Professional & knowledgeable staff. Food was on point!
query: ['grill', 'chees']
Similarity: 11.79193115234375
Their truffle fries are good but it's the super line cut kind and you don't get
Similarity: 11.763312339782715
"roasted chanterelle mushrooms, truffle cheese: DELICIOUS."
Similarity: 11.622526168823242
They have a truffle cheese that should be accompanied by a cigarette.
```

- 1.3. For the first three queries, analyze the relation between relevance and cosine similarity score: is a high score document more relevant to the query?

We observe that a higher similarity score provides a more relevant document to the query, at 0, we don't even observe any words of the query in the document. the higher the scoring, the more likely for more words of the query to be in the document but it isn't always the case. When the scoring is near 0, the vectors are nearly orthogonal, which states a lack of similarity.

2. TF-IDF (40pts)

- 2.1. Paste your implementation of document representation and cosine similarity calculation. Report the training time of the word embedding model.

Training time: 0.8700037002563477s

```
from gensim.models import TfidfModel
from gensim.corpora import Dictionary
```

```
dct = Dictionary(dataset) # fit dictionary
corpus = [dct.doc2bow(line) for line in dataset]
```

```
start = time.time()
tfidf_model = TfidfModel(corpus)
end = time.time()
print(f"TfidfModel training time: {end - start}")
tfidf_model.save("tfidf_model.model")
```

```
from gensim import similarities
```

```
tfidf_corpus = tfidf_model[corpus]
print("Top 3 document for each query:")
for query in processed_queries:
    corpus_idx = 0
    doc_similarity = []
    query_bow = dct.doc2bow(query)
    index = similarities.SparseMatrixSimilarity(tfidf_corpus, num_features=len(
    similarity_scores = index[tfidf_model[query_bow]]
```

```

print(f'query: {query}')
```

```

for idx, similarity in sorted(enumerate(similarity_scores), key=lambda x: x[1]):
    print(f'Similarity: {similarity}')
    print(dataset_content[idx])
```

- 2.2. For the top 3 documents of each query, print the document and its cosine similarity score.

```

query: ['gener', 'chicken']
Similarity: 0.5499180555343628
Great atmosphere and delicious food with generous servings!
Similarity: 0.5499180555343628
Great atmosphere and delicious food with generous servings!
Similarity: 0.39725804328918457
Amazing chicken!
query: ['fri', 'chicken']
Similarity: 0.8193947672843933
The Fried Chicken is amazing!!
Similarity: 0.6181540489196777
The fried chicken taste great and the fries were very fresh and the customer se
Similarity: 0.6069390773773193
I went here for brunch - the fried chicken and waffles was delicious!!! If anyt
query: ['bbq', 'sandwich']
Similarity: 0.7988479733467102
Very good bbq.
Similarity: 0.7820755243301392
great bbq
Similarity: 0.7820755243301392
great bbq!
query: ['mash', 'potato']
Similarity: 0.7255823612213135
A place that has mashed potatoes as pizza topping! need I say more?
Similarity: 0.6991888880729675
Mashed potatoes on pizza? Sign me up!
Similarity: 0.6764199137687683
5 stars for the ill topping selection: MASHED POTATOES.
query: ['grill', 'shrimp', 'salad']
Similarity: 0.5165484547615051
Great atmosphere! The sea bass looked good, but I ordered a salad with grilled
Similarity: 0.4595300853252411
Grill oyster
Similarity: 0.45436424016952515
Grilled corn!
query: ['lamb', 'shank']
Similarity: 0.5350815057754517
```

Last night I went here with my favorite eating buddy. We both love Tapas (our
Similarity: 0.5084143280982971
Visited Chicago last year and stopped by this tapas bar . . and I loved it! I
Similarity: 0.472200483083725
wonderful food, not too pricey for what you're getting. ordered the pork shank,
query: ['pepperoni', 'pizza']
Similarity: 0.8301189541816711
I really wanted to like this pizza, especially after all the glowing reviews. A
Similarity: 0.7200928926467896
5* Deep Dish Pepperoni
Similarity: 0.6721419095993042
This pizza was freakin good! Nice brick oven style pizza, we got pepperoni piz
query: ['brussel', 'sprout', 'salad']
Similarity: 0.8049801588058472
Best brussel sprouts I've ever experienced - who gets excited about a brussel s
Similarity: 0.7307406663894653
Solid food all around. Good wine. Good steaks, Good sides. Not much more to s
Similarity: 0.6625710725784302
almost the best steak i've ever had in my life. amazing brussel sprouts as well
query: ['friendli', 'staff']
Similarity: 0.7867393493652344
Long wait good food friendly staff
Similarity: 0.7105895280838013
Great place to eat with friends friendly staff and food is great
Similarity: 0.6909550428390503
Great food, reasonable prices, friendly staff
query: ['grill', 'chees']
Similarity: 0.7510269284248352
All I have to say is try the grilled cheese. I think about it all the time. S
Similarity: 0.6265907287597656
Grill oyster
Similarity: 0.6195468306541443
Grilled corn!

- 2.3. For the first three queries, analyze the relation between relevance and cosine similarity score: is a high score document more relevant to the query?

We observe that a higher similarity score provides a more relevant document to the query, at 0, we still don't observe the query words in the document at a low score. The higher the scoring, the more likely for more words of the query to be in the document. This is the same for Word2Vec, but in this case, we observe that the query words take a bigger portion of the document.

- 1.4. Compare TF-IDF with averaged word2vec, which model gives a better document representation and query representation? Why?

I believe word2vec provides a better document and query representation in general

because the query tends to be related and when semantic relationships between words are captured it would better address the query. In TF-IDF we could guarantee that the document share a higher importance in the words given in the query as we don't take semantic meaning into account but just tf-idf values.

In conclusion, queries that are longer and have semantic meanings would do better with word2vec, whereas keyword based queries would perform better in tf-idf.